

To complete the task, I first had to understand exactly what a fiscal printer is, what it works with and how, what its role is in POS systems, as well as the basics of POS systems themselves.

Next, I familiarized myself with the model referred to in the task, i.e. Deon, as well as with the attached documentation.

Of the variants currently available for purchase, Deon Online is the available version, so I assumed that this is the variant referred to in the task.

The task description states that the protocol included in the attachment should be used, so the entire task is based on that protocol.

The attached documentation describes the classic protocol rather than the newer XML protocol, so the classic protocol is used.

The Deon Online printer supports three command styles: Novitus, Novitus-compatible, and Posnet.

Since Novitus mode is the default and is also the one described in the attached documentation, the application uses this mode.

The task description states that the application should accept a product name, price, VAT rate, and an optional fixed-amount discount as its list of parameters.

It says "product name," not "product names," and there is no mention of quantity, so I assume that the task concerns a single product/item with a quantity of 1.

According to the provided documentation, the PTU field (VAT rate) is of type CHARACTER and accepts a group identifier, so I assume this format for the VAT rate input and assume that the printer already has PTU groups configured and that the user knows which group corresponds to which VAT rate.

A serial interface was assumed as the standard interface, in accordance with the "Interface Description" section (PDF page 295).

The connection parameters were selected according to the information provided on the same page of the documentation.

The protocol always assumes no parity, 8 data bits, and one stop bit, so these values were treated as fixed.

The same applies to the baud rate, because values other than 9600 are available only for Quarto/Delio.

I assume software XON/XOFF handshaking in accordance with Table 11.1. It is always supported, unlike RTS/CTS, which requires the corresponding physical signals to be connected correctly or properly emulated. Software handshaking works without problems for both RS232 and USB.

The task states that the program should be simple, so extending it beyond what was requested would go against that requirement. Therefore, I omit features such as reading the currently configured PTU rates or processing errors returned by the printer.

After establishing the initial assumptions required to complete the task, I moved on to the next stages.

CMakeFile:

At the beginning of the application development process, the project assumptions are passed to CMake.

To ensure that the program can be compiled correctly, I specified a minimum CMake version of 3.16, which is sufficiently modern while still being widely available.

Only the C++ language is enabled, in accordance with the task requirements.

The C++17 standard is selected and explicitly required. If the compiler cannot satisfy this requirement, the application will not compile, preventing accidental use of an older standard.

EXTENSIONS are also disabled to prevent the use of compiler-specific extensions that could cause differences between compilers.

These steps were taken to ensure consistent reproducibility and predictable program behavior.

At a later stage, after creating the tests, I extended the CMakeFile to make testing easier.

In the protocol, there are transaction printing / receipt opening modes associated with the number of items: block mode, when the number of receipt items is known in advance, and online mode, when the number of items is not known in advance (PDF page 89).

In this task, we know the number of receipt items in advance, so block mode could be used. In practice, however, online mode is used in the vast majority of cases where the final number of receipt items is not known in advance, so I decided to use online mode.

The “begin transaction” command has three variants depending on the type of receipt that is to be issued:

a standard receipt, a receipt with additional lines of text, or a receipt containing NIP / REGON / PESEL identification data.

There is no command that explicitly informs the printer which receipt type is going to be printed. The printer recognizes the receipt type automatically based on the data format passed through the “begin transaction” command.

For the purpose of this simple application, I selected a standard receipt, which uses the least complex variant of the “begin transaction” command.

To prevent calculation errors when calculating monetary values — both for correctness and to avoid rejection by the printer in the event of a calculation mismatch, as described in the protocol — all calculations are performed using integers expressed in grosze. After all calculations have been completed, the values are converted back into the złoty.grosz format.

The program is supposed to accept only the following parameters: product name, price, VAT rate, and an optional fixed-amount discount.

This would mean that either the serial port would have to be hardcoded, requiring recompilation whenever the port changes, which would be inconvenient; or support for storing the port in a file would have to be added, which would slightly complicate the application; or the port could be supplied as an additional parameter.

Since the application is supposed to remain simple, but I do not want it to be inconvenient to use, I chose a compromise and added support for specifying the port through an optional additional

command-line parameter when launching the application. If no port is specified, /dev/ttyUSB0 is used by default.

Interface usage:

without specifying a port — the default /dev/ttyUSB0 will be used

```
./deon_printer_app "NAZWA" CENA PTU
```

```
./deon_printer_app "NAZWA" CENA PTU [RABAT kwotowy]
```

with a specified port:

```
./deon_printer_app --port=/dev/ttyS0 "NAZWA" CENA PTU
```

```
./deon_printer_app --port=/dev/ttyS0 "NAZWA" CENA PTU [RABAT kwotowy]
```

If there are problems with file permissions, use:

```
chmod 755 deon_printer_app
```

(everyone can read, everyone can execute, only the owner can write)

Testing the program:

The program can be tested without a printer by changing the DRY_RUN value in main.cpp to true:

```
constexpr bool DRY_RUN = true;
```

This causes the output to be written to the console instead of the port and also skips the port presence check.

The program also contains unit tests, and their usage is described in README.md.

Completing the task

I started by carefully reading and understanding the task description in order to fully understand the requirements and the essence of the task.

I then proceeded to collect the requirements: a C++ application for Linux, launched from the command line, which issues a receipt using a Deon Online fiscal printer over RS232 / USB CDC according to the described protocol.

Next, I familiarized myself with the provided documentation, the problem domain, and the terminology:

what a fiscal printer is, what a cash register is and how they differ,

what POS systems are, how the components within them relate to one another and how they operate, as well as the description of the device itself.

Based on this, I identified the main tasks: parsing CLI arguments, implementing receipt logic — price, PTU, discount — and implementing the technical communication with the printer through the serial port, including frame construction.

From the documentation I determined, among other things:

the communication interface — RS232, which under Linux, thanks to the standard driver, can be used in a similar way to USB CDC, which determines how the connection to the device should be established,

I selected XON/XOFF software handshaking as the most universal and reliable approach for both RS232 and USB CDC,

connection parameters — baud rate 9600, 8 data bits, no parity, 1 stop bit,

the required commands and the rules governing their operation,

information concerning the checksum,

as well as the fact that:

the program must support both a receipt with a fixed-amount discount and a receipt without a fixed-amount discount, where the discount parameter is absent. It must determine which variant is being used based on the number of supplied parameters and select the appropriate behavior,

it must perform calculations by parsing złoty and grosz values into integer values expressed in grosze, and perform all calculations in grosze, including quantity and the optional fixed-amount discount. In this task, the quantity is 1,

there must be no calculation errors. The calculations must use integer values expressed in grosze and, once completed, the values must be converted back from grosze to the złoty.grosz representation,

the output format must always be złoty.grosz, and the input should use the same format. I also added support for values containing only one digit after the decimal separator — for example, 5.2 is interpreted as 5.20 — while 5.234 returns an error,

the application must calculate the CRC according to the protocol description and insert it in the appropriate locations defined by the protocol in the PDF,

the VAT rate is supplied according to the PTU groups configured in the printer.

I decided on the architecture by establishing the requirements and considering them together with the input data, output data, and the transformations that need to take place between them.

I defined and documented, step by step, what the program should do — in other words, I described the application flow and the individual tasks it must perform.

I documented the requirements imposed by the documentation and protocol, such as the need for exact calculations for the receipt to be accepted, which led to the requirement to use integer arithmetic.

I also identified the protocol commands required to complete the task.

I planned an architecture that divides the application into modules corresponding to the application flow and to the individual tasks performed by the application.

I planned the layers that should make up the application.

Initially, I tried to get as far as possible based on logic, my existing knowledge, familiarity with SOLID principles, and intuition: determine the requirements, inputs and outputs, as many processing steps as possible, what exactly the program should do, what data it should accept, what exactly it should do with that data, which commands should be sent, what the program flow should look like, how the

connection to the printer should be established, and what the communication interface and parameters should be according to the protocol.

Later, I searched the literature for established patterns that matched or resembled my own design concept.

A significant part of what I had developed independently matched existing documented patterns. Among other sources, I referred to *Design Patterns: Elements of Reusable Object-Oriented Software*.

I refreshed my knowledge of software architecture, design patterns, the tasks involved in creating such an application, and how such problems should be approached according to established engineering practices.

I developed the architecture based on the patterns that most closely matched the way I had independently envisioned the application.

From this, I established the following architecture:

A simple layered architecture based on the Transaction Script approach, with a thin façade over the protocol and an adapter for the hardware layer.

Based on this, I divided the program into layers and determined what each layer should do with the data.

I adopted the principle that higher layers should use only lower layers and that dependencies should flow in a single direction. This keeps the receipt logic separated from the implementation details of RS232 and Linux.

I determined which mechanisms should be used, for example RAII to guarantee reliable resource release.

Next, I created the application skeleton in order to start implementing the code, relationships, and execution flow.

I began by creating the files and establishing the includes and headers.

Architecture description

The code was written in the same general way I have typically written programs so far — following the flow of the application — but with several simple, classic patterns used together.

1. Main style: Transaction Script

This matches the flow exactly:

main() → read arguments → calculate amounts → build frames → send them sequentially

In Fowler's terminology, this is called a Transaction Script — one simple “transaction script” responsible for executing one specific use case, which in this case is issuing a receipt containing one item and an optional discount.

I used a Transaction Script-style approach: the main function performs one business scenario — issuing a simple receipt — in an organized, step-by-step manner.

2. Module structure: Layered Architecture

Layered Architecture:

- “input” layer → main — argument parsing,
- “logic” layer → receipt_logic, money_utils — calculations using amounts expressed in grosze,
- “protocol” layer → novitus_protocol — construction of command bodies and frames,
- “infrastructure” layer → SerialPort implemented using termios.

The code is divided into layers: the input layer (CLI), business logic (amounts expressed in grosze), the printer protocol layer (frame formatting, CRC), and the infrastructure layer (the serial port implemented using termios).

As a result, main only orchestrates these layers.

3. Simple “pattern” around the printer: Small Façade + Adapter

- the function printSimpleReceipt(const ReceiptContext&, SerialPort&) acts as a small Façade over the protocol — main does not need to know about \$h / \$l / \$Y / \$e / \$z; it only calls one function,
- the SerialPort class effectively acts as an Adapter for POSIX:

- o it hides open / tcsetattr / write,

- o and provides a simple writeAll method.

I wrapped access to the printer in a simple SerialPort class, which adapts the low-level POSIX interface (termios, write) into a simple writeAll call.

On top of that, I added a small “façade” — the printSimpleReceipt function — which hides the protocol details, such as command order and frame construction, from the rest of the application.

How I designed this program

1. First, understanding the domain

I started by understanding the domain: what a fiscal printer is, what role it plays in a POS system, and how the Novitus protocol works at the frame and command level.

2. Then, dividing responsibilities into layers

Based on this, I divided the application into layers:

- main — obtaining parameters from the command line,
- amount logic — parsing values such as "12.34" into grosze and performing simple calculations,
- printer protocol — constructing command bodies and frames with CRC,
- infrastructure — handling the serial port under Linux using termios.

3. Choosing the Transaction Script style

I designed the main flow in main() using the Transaction Script style: one function orchestrates the individual steps in a fixed sequence — read parameters, calculate amounts, build frames, and send them sequentially to the printer.

4. Thin façade + adapter

To simplify the use of the protocol, I added a small façade:

```
printSimpleReceipt(const ReceiptContext&, SerialPort&)
```

which knows the command sequence and the details of the protocol.

I wrapped access to the device itself in the SerialPort class, which acts as an adapter between low-level POSIX operations (open, termios, write) and the simple writeAll call.

5. Iterative implementation of the layers

I implemented the application iteratively: first, I created a skeleton containing empty functions and a fake SerialPort that printed to stdout.

Then I added parsing of monetary values into grosze, followed by frame construction and CRC calculation, and finally the actual serial port configuration and tests using a virtual serial port.

The program was developed in the spirit of SOLID principles while also taking issues such as memory leaks into account.

At the very end, as a self-code-review exercise, I added comments to every line.

I am aware that there are too many comments and that under normal circumstances I use significantly fewer of them. Originally, there were far fewer comments, but the current amount is mainly the result of an exercise I performed for myself.

At the same time, they may make the program easier to understand, so I decided to leave them in place.

I experimented with their placement in the code so that they would not reduce readability while still providing useful information.

Since the task has the characteristics of an exercise, I considered this acceptable in this particular case, although I admit that I repeatedly considered reducing the number of comments. In the end, I decided that for an exercise of this kind, their presence was acceptable.

Protocol command sequence for one product with an optional fixed-amount discount applied to the entire receipt

1. 3.4.1. Beginning of transaction — online receipt

Command: 0\$h — first variant of the parameter structure

Meaning: opens the receipt in online mode (number of items = 0), i.e. ONLINE mode.

2. 3.4.2. Receipt line — issuing a receipt item

(one product, quantity 1)

Command: 1\$I — first variant of the parameter structure

Fields:

- line number = 1,
- product name = NAZWA parameter,

- quantity = 1,
- PTU = PTU parameter (A..G),
- price = CENA parameter,
- gross amount = price × quantity; for a quantity of 1, this is equal to CENA.

Variant without an item-level discount.

3. 3.4.15. Receipt discount/surcharge applied to the subtotal

— optional fixed-amount discount applied to the entire receipt subtotal

Command: \$Y — “Receipt discount/surcharge applied to the subtotal”

Discount type = 3 — fixed-amount discount

Fields:

- discount type = 3,
- description number = e.g. 0 — without a predefined description,
- subtotal = gross value of all items before the discount; for one product, this is the gross amount from 1\$,
- discount = discount amount provided as the application's optional parameter,
- discount description — optional text, e.g. "RECEIPT DISCOUNT", or an empty string.

4. 3.4.9. Standard transaction confirmation

(receipt closing)

Command: 1\$e — “Standard transaction confirmation”

Fields:

- action = 1 — confirm,
- discount = 0 — the discount has already been handled using \$Y,
- cashier name — a fixed or empty string,
- payment = amount to be paid after the discount (subtotal – fixed-amount discount),
- total = the same amount as the payment.

All of the above commands are transmitted using protocol frames consisting of the <ESC>P header, fields separated by separators, an XOR control byte, and the <ESC>\ terminator.